

Agenda

- Upcasting & Downcasting
- Final Keyword
- Object class
 - toString()
 - equals()
- ~~Abstract Method and Class~~
- ~~Interface~~

Rules of method overriding in Java

1. Each method in Java can be overridden unless it is private, static or final.
 2. Sub-class method must have same or wider access modifier than super-class method.
 3. Arguments of sub-class method must be same as of super-class method.
 4. The return-type of sub-class method can be same or sub-class of the super-class's method's return-type. This is called as "covariant" return-type.
 5. Checked exception list in sub-class method should be same or subset of exception list in super-class method.
- If these rules are not followed, compiler raises error or compiler treats sub-class method as a new method.
 - Java 5.0 added @Override annotation (on sub-class method) informs compiler that programmer is intending to override the method from the super-class.
 - @Override checks if sub-class method is compatible with corresponding super-class method or not (as per rules). If not compatible, it raise compile time error.
 - Note that, @Override is not compulsory to override the method. But it is good practice as it improves readability and reduces human errors.

Upcasting

- Assigning sub-class reference to a super-class reference.
- Sub-class "is a" Super-class, so no explicit casting is required.
- Using such super-class reference, only super-class methods inherited into sub-class can be called. This is "Object slicing".
- Using such super-class reference, super-class methods overridden into sub-class can also be called.

Polymorphism

- poly = Many , morphism = Forms
- It has two types
 1. compile time
 - implemented using method overloading
 - Compiler can identify which method to be called at compile time depending on types of arguments. This is also referred as "Early binding".
 2. Runtime
 - implemented using method overriding

- The method to be called is decided at runtime depending on type of object.
- This is also referred as "Late binding" or "Dyanmic method dispatch".

```
public class Program01 {  
    public static void main(String[] args) {  
        Employee e = null;  
        // e = new Employee();  
        // e = new Manager(); // upcasting  
        e = new Salesman(); // upcasting  
  
        // Dynamic Method Dispatch - Runtime Polymorphism  
        e.accept();  
        e.display();  
    }  
}
```

Downcasting

- Assigning super-class reference to sub-class reference.
- Every super-class is not necessarily a sub-class, so explicit casting is required.

```
class Employee {  
    private int id;  
    private String name;  
    private double salary;  
  
    public void accept() {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter id = ");  
        id = sc.nextInt();  
        System.out.print("Enter name = ");  
        name = sc.next();  
        System.out.print("Enter salary = ");  
        salary = sc.nextDouble();  
    }  
  
    public void display() {  
        System.out.println("id = " + id);  
        System.out.println("Name = " + name);  
        System.out.println("Salary = " + salary);  
    }  
}  
  
class Manager extends Employee {  
    private double bonus;  
  
    public void accept() {  
        Scanner sc = new Scanner(System.in);  
        super.accept();  
        System.out.print("Enter bonus = ");  
    }  
}
```

```
        bonus = sc.nextDouble();
    }

    public void display() {
        super.display();
        System.out.println("Bonus = " + bonus);
    }
}

class Salesman extends Employee {
    private int salesCount;
    private double commission;

    public void accept() {
        Scanner sc = new Scanner(System.in);
        super.accept();
        System.out.print("Enter sales count = ");
        salesCount = sc.nextInt();
        System.out.print("Enter commission = ");
        commission = sc.nextDouble();
    }

    public void display() {
        super.display();
        System.out.println("SalesCount = " + salesCount);
        System.out.println("Commission = " + commission);
    }

    public void calculateTotalCommission() {
        double total = salesCount * commission;
        System.out.println("Total Commission - " + total);
    }
}

public class Program01 {
    public static void main(String[] args) {
        Employee e = null;
        // e = new Employee();
        // e = new Manager();
        e = new Salesman();

        e.accept();
        e.display();

        // e.calculateTotalCommission(); // NOT possible - Object Slicing
        Salesman s = (Salesman) e; // Downcasting
        s.calculateTotalCommission(); // OK
    }
}
```

```

public class Program01 {
    public static void main(String[] args) {
        Employee e = null;
        // e = new Employee();
        e = new Manager();

        e.accept();
        e.display();

        Salesman s = (Salesman) e; // ClassCastException
        // As upcasting is not done i.e the salesman object is not stored in employee
        // reference
    }
}

```

instanceof operator

- Java's instanceof operator checks if given reference points to the object of given type (or its sub-class) or not. Its result is boolean.
- Typically "instanceof" operator is used for type-checking before down-casting.

```

```JAVA
Employee e = null;
// e = new Employee();
// e = new Manager();
e = new Salesman();

e.accept();
e.display();

if (e instanceof Salesman) {
 Salesman s = (Salesman) e; // Downcasting
 s.calculateTotalCommission();
}

```

## final keyword

- In Java, const is reserved word, but not used.
- Java has final keyword instead.
- It can be used for
  - variables
  - fields
  - methods
  - class

## final variable

- if variables and fields are made final, they cannot be modified after initialization.

```
public static void main1(String[] args) {
 final int num1 = 10;
 // num1 = 20; // NOT OK
}

public static void main2(String[] args) {
 final int num1; // final local can be declared without initialization
 num1 = 10; // OK
 // num1 = 20; // NOT OK - Once initialized cannot be reassigned
}
```

## final field

- final fields of the class must be initialized using any of the following below
  - field initializer
  - object initializer
  - constructor

```
class Circle {
 // final double PI = 3.14;
 final double PI;

 {
 // PI = 3.14;
 }

 public Circle() {
 PI = 3.14;
 }
}
```

## final Method

- If implementation of a super-class method is logically complete, then the method should be declared as final.
- Such final methods cannot be overridden in sub-class. Compiler raise error, if overridden.
- But final methods are inherited into sub-class i.e. The super-class final methods can be invoked in sub-class object (if accessible).
- eg: getClass(), wait(), notify() are the few methods from Object class which are declared as final.

## final Class

- If implementation of a super-class is logically complete, then the class should be declared as final.
- The final class cannot be extended into a sub-class. Compiler raise error, if inherited.
- Effectively all methods in final class are final methods.
- Examples of final classes
  - java.lang.Integer (and all wrapper classes)

- java.lang.String
- java.lang.System

## Object class

- Non final and non-abstract class declared in java.lang package.
- In java, all the classes (not interfaces) are directly or indirectly extended from Object class.
- In other words, Object class is ultimate base class/super class hierarchy.
- Object class is not inherited from any class or implement any interface.
- It has a default constructor. Object o = new Object();
- Object class methods (read docs)
  - public Object();
  - public native int hashCode();
  - public boolean equals(Object);
  - protected native Object clone() throws CloneNotSupportedException;
  - public String toString();
  - protected void finalize() throws Throwable;
  - public final native Class<?> getClass();
  - public final native void notify();
  - public final native void notifyAll();
  - public final void wait() throws InterruptedException;
  - public final native void wait(long) throws InterruptedException;
  - public final void wait(long, int) throws InterruptedException;

## toString() method

- it is a non final method of object class
- To return state of Java instance in String form, programmer should override toString() method.
- The result in toString() method should be a concise, informative, and human-readable.
- It is recommended that all subclasses override this method.

```
public class Date {
 private int day;
 private int month;
 private int year;

 public Date() {
 }

 public Date(int day, int month, int year) {
 this.day = day;
 this.month = month;
 this.year = year;
 }
}
```

```
@Override
public String toString() {
 return "Date = "+this.day+"/"+this.month+"/"+this.year;
}

public static void main(String[] args) {
 Date d1 = new Date();
 Date d2 = new Date(1, 1, 2001);
 System.out.println("d1 = " + d1);
 System.out.println("d2 = " + d2);
 // date value will be only printed if toString method is overridden otherwise it
 prints the state as fully qualified classname @ hashCode
}
```

## equals() method

- It is non final method of object class
- To compare the object contents/state, programmer should override equals() method.
- This equals() must have following properties:
  - Reflexive: for any non-null reference value x, x.equals(x) should return true.
  - Symmetric: for any non-null reference values x and y, x.equals(y) should return true if and only if y.equals(x) returns true.
  - Transitive: for any non-null reference values x, y, and z, if x.equals(y) returns true and y.equals(z) returns true, then x.equals(z) should return true.
  - Consistent: for any non-null reference values x and y, multiple invocations of x.equals(y) consistently return true or consistently return false, provided no information used in equals comparisons on the objects is modified.
- For any non-null reference value x, x.equals(null) should return false.
- It is recommended to override hashCode method along when equals method is overridden.

```
public class Date {
 private int day;
 private int month;
 private int year;

 public Date() {
 }

 public Date(int day, int month, int year) {
 this.day = day;
 this.month = month;
 this.year = year;
 }

 @Override
 public String toString() {
 return "Date = "+this.day+"/"+this.month+"/"+this.year;
 }
}
```

```
}

@Override
public boolean equals(Object obj) {
 if (obj == null)
 return false;
 else if (this == obj)
 return true;
 else if (obj instanceof Date) {
 Date d = (Date) obj;
 if (this.day == d.day && this.month == d.month && this.year == d.year)
 return true;
 }
 return false;
}

public static void main(String[] args) {
 Date d1 = new Date(1, 1, 2001);
 System.out.println("d1 = " + d1);

 Date d2 = new Date(1, 1, 2001);
 System.out.println("d2 = " + d2);

 if (d1 == d2)
 System.out.println("Date is equal");
 else
 System.out.println("Date is not equal");

 if (d1.equals(d2))// return true only when equals method is overridden
 System.out.println("Date is equal");
 else
 System.out.println("Date is not equal");
}
}
```

## Assignment

- Create a class called as Time with fields hr,min,sec. Implement the toString() and equals() , test all the cases.